

TP 10 : Apache Cassandra

Université / École :

- Université Ibn Tofail, Kénitra

Encadrement pédagogique :

- Pr. Hajar Filali

Rédigé par :

- Mohammed Jebrou
- Hajar Farid

Année universitaire :

- 2025–2026

Introduction à Apache Cassandra

Apache Cassandra est une base de données NoSQL distribuée conçue pour gérer de grands volumes de données tout en garantissant une haute disponibilité et une excellente tolérance aux pannes. Grâce à son architecture décentralisée, Cassandra permet de stocker et de traiter efficacement les données sur plusieurs nœuds sans point unique de défaillance.

Dans ce travail pratique, nous avons installé et configuré Apache Cassandra, puis exploré les principales fonctionnalités du système à travers l'utilisation du langage CQL (Cassandra Query Language). Les différentes manipulations réalisées ont permis de mettre en œuvre la création de structures de stockage, la gestion des données, l'utilisation du mécanisme TTL ainsi que l'étude des concepts de modélisation et de cohérence propres à Cassandra.

Objectifs du TP

À l'issue de ce travail pratique, nous serons capables de :

- Installer et configurer Apache Cassandra dans un environnement Linux.
- Comprendre l'architecture et les principes de fonctionnement d'une base de données NoSQL distribuée.
- Utiliser l'outil CQLSH pour administrer une base Cassandra.
- Créer et gérer des Keyspaces et des tables.
- Réaliser les opérations de manipulation des données (insertion, consultation, mise à jour et suppression).
- Utiliser le mécanisme TTL pour gérer les données temporaires.
- Concevoir des tables adaptées aux besoins des requêtes grâce à la modélisation orientée requêtes.
- Comprendre et expérimenter les différents niveaux de cohérence proposés par Cassandra.
- Maîtriser les opérations fondamentales d'administration et d'exploitation d'une base de données Cassandra.

Installation du Cassandra

1. Téléchargement et extraction des fichiers

Cette étape consiste à télécharger l'archive officielle d'Apache Kafka puis à l'extraire dans le répertoire de travail. Les fichiers obtenus contiennent les exécutables, les bibliothèques et les fichiers de configuration nécessaires au fonctionnement de Kafka.

```
wget
) wget https://downloads.apache.org/cassandra/4.1.11/apache-cassandra-4.1.11-bin.tar.gz
--2026-06-01 22:52:37-- https://downloads.apache.org/cassandra/4.1.11/apache-cassandra-4.1.11-bin.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 95.216.224.44, 88.99.208.237, 2a01:4f8:10a:39da::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|95.216.224.44|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 53640694 (51M) [application/x-gzip]
Saving to: 'apache-cassandra-4.1.11-bin.tar.gz'

apache-cassand  0%[                  ] 78.00K  42.7KB/s
```

```
mohammed@ROG-Strix:~
) tar -xvf apache-cassandra-4.1.11-bin.tar.gz -C ~/cassandra --strip-components=1
apache-cassandra-4.1.11/bin/
apache-cassandra-4.1.11/conf/
apache-cassandra-4.1.11/conf/triggers/
apache-cassandra-4.1.11/doc/
apache-cassandra-4.1.11/doc/cql3/
apache-cassandra-4.1.11/lib/
apache-cassandra-4.1.11/lib/sigar-bin/
apache-cassandra-4.1.11/pylib/
```

2. Configuration des variable d'environnement

```
nvim $HOME/.zsh//env.zsh
mohammed@ROG-Strix:~/Desktop/COUR... nvim $HOME/.zsh//env.zsh
50
51
52 export CASSANDRA_HOME=$HOME/cassandra
53 export PATH=$PATH:$CASSANDRA_HOME/bin
54
55 |
```

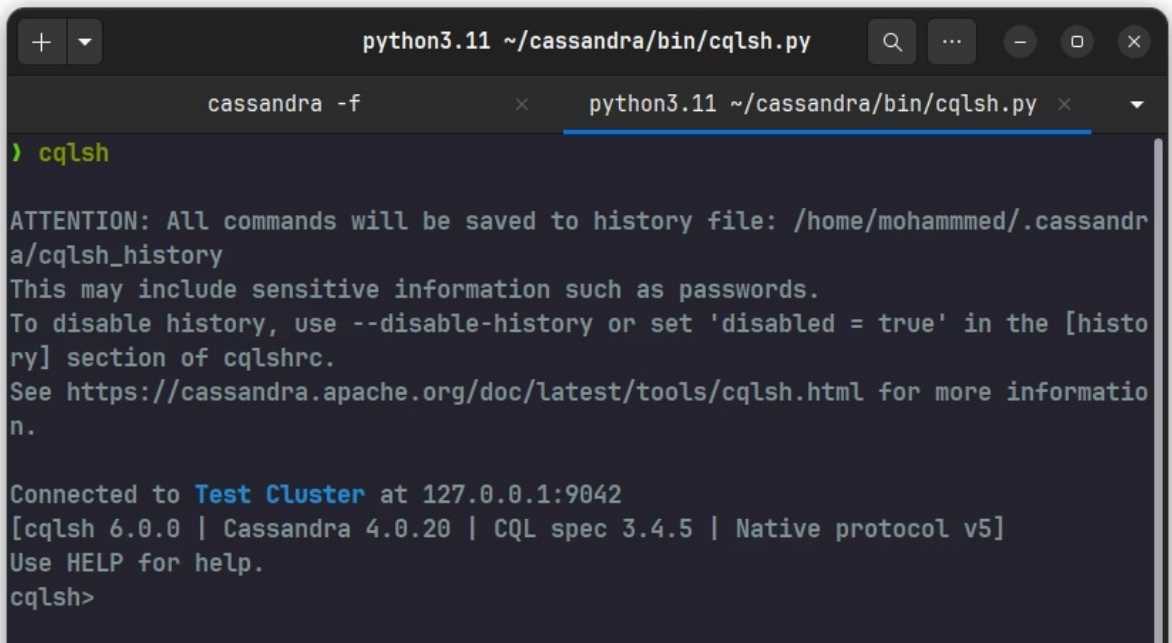
3. Démarrer Cassandra

```
cassandra -f
> cassandra -f
CompilerOracle: dontinline org/apache/cassandra/db/Columns$Serializer.deserializeLargeSubset (Lorg/apache/cassandra/io/util/DataInputPlus;Lorg/apache/cassandra/db/Columns;I)Lorg/apache/cassandra/db/Columns;
CompilerOracle: dontinline org/apache/cassandra/db/Columns$Serializer.serializeLargeSubset (Ljava/util/Collection;ILorg/apache/cassandra/db/Columns;ILorg/apache/cassandra/io/util/DataOutputPlus;)V
CompilerOracle: dontinline org/apache/cassandra/db/Columns$Serializer.serializeLargeSubsetSize (Ljava/util/Collection;ILorg/apache/cassandra/db/Columns;I)I
CompilerOracle: dontinline org/apache/cassandra/db/commitlog/AbstractCommitLogSegmentManager.advanceAllocatingFrom (Lorg/apache/cassandra/db/commitlog/CommitLogSegment;)V
CompilerOracle: dontinline org/apache/cassandra/db/transform/BaseIterator.tryGetMoreContents ()Z
CompilerOracle: dontinline org/apache/cassandra/db/transform/StoppingTransformation.stop ()V
CompilerOracle: dontinline org/apache/cassandra/db/transform/StoppingTransformation.stopInPartition ()V
CompilerOracle: dontinline org/apache/cassandra/io/util/BufferedDataOutputStreamPlus.doFlush (I)V
CompilerOracle: dontinline org/apache/cassandra/io/util/BufferedDataOutputStreamPlus.writeSlow (JI)V
CompilerOracle: dontinline org/apache/cassandra/io/util/RebufferingInputStream.readPrimitiveSlowly (I)J
```

4. Vérification avec nodetool status

```
mohammed@ROG-Strix:~/cassandra
cassandra -f
> nodetool status
Datacenter: datacenter1
=====
Status=Up/Down
// State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens  Owns (effective)  Host ID
--  --
UN 127.0.0.1 104.35 KiB 16 100.0% 68376944-f3c0-44cc-bbb7-ac1bff2caa8a rack1
cassandra >
```

5. Lancer le shell Cassandra



```
python3.11 ~/cassandra/bin/cqlsh.py
cassandra -f  python3.11 ~/cassandra/bin/cqlsh.py
> cqlsh

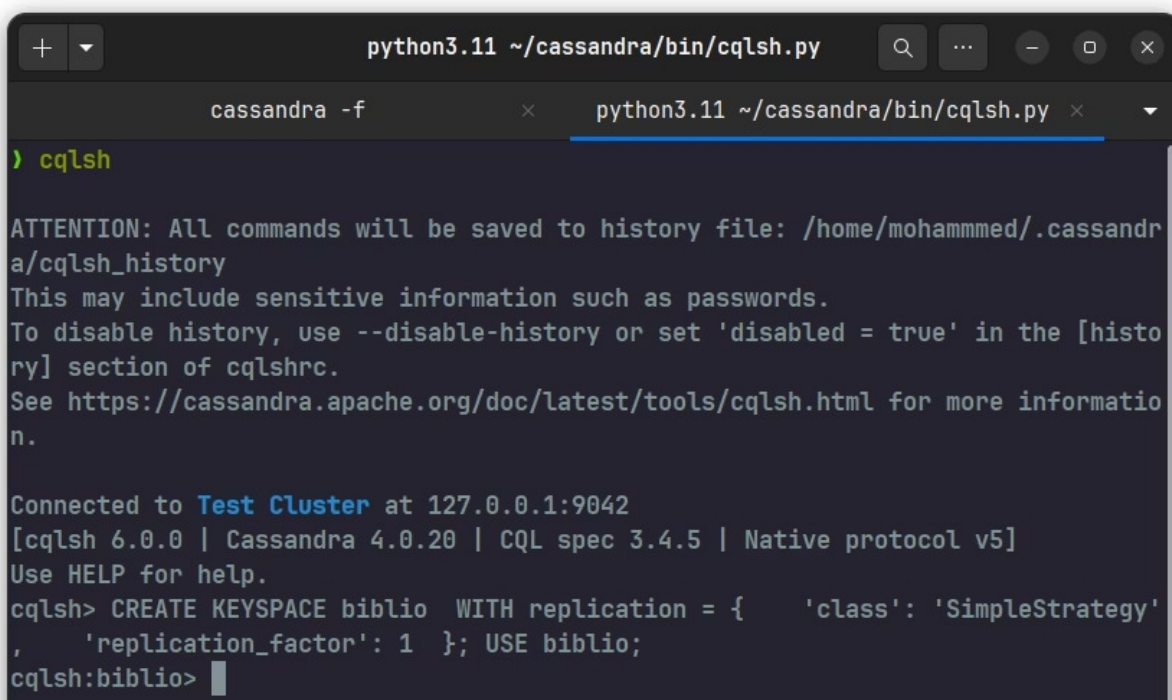
ATTENTION: All commands will be saved to history file: /home/mohammed/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.0.0 | Cassandra 4.0.20 | CQL spec 3.4.5 | Native protocol v5]
Use HELP for help.
cqlsh>
```

6. Création du Keyspace et des Tables

6.1 Création du Keyspace biblio

Un Keyspace représente l'espace de stockage principal dans Cassandra. Il est comparable à une base de données dans les systèmes relationnels. Dans cette étape, un Keyspace nommé `biblio` est créé avec une stratégie de réplication `SimpleStrategy` et un facteur de réplication égal à 1 afin de stocker les futures tables de l'application.



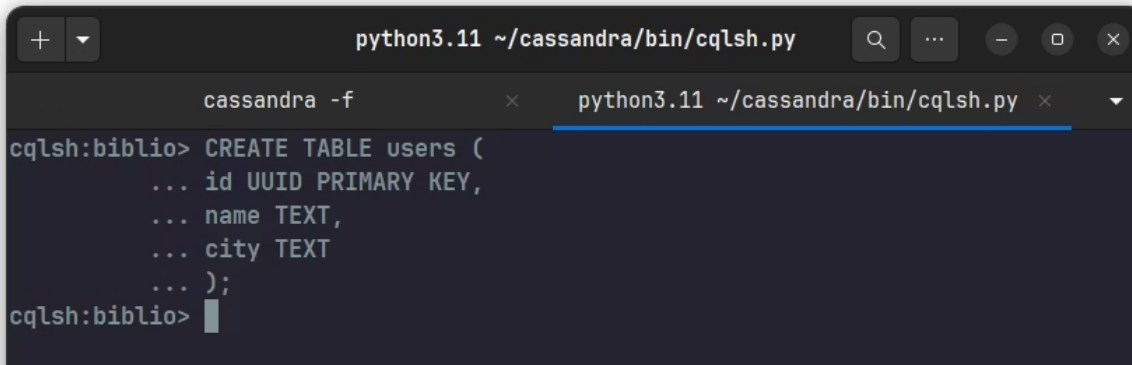
```
python3.11 ~/cassandra/bin/cqlsh.py
cassandra -f  python3.11 ~/cassandra/bin/cqlsh.py
> cqlsh

ATTENTION: All commands will be saved to history file: /home/mohammed/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.0.0 | Cassandra 4.0.20 | CQL spec 3.4.5 | Native protocol v5]
Use HELP for help.
cqlsh> CREATE KEYSPACE biblio WITH replication = { 'class': 'SimpleStrategy'
, 'replication_factor': 1 }; USE biblio;
cqlsh:biblio>
```


6.2 Création de la table users

Après la création du Keyspace `biblio`, une table nommée `users` est créée afin de stocker les informations relatives aux utilisateurs. Cette table contient trois attributs : un identifiant unique (`id`) de type `UUID`, le nom de l'utilisateur (`name`) et sa ville (`city`). La clé primaire `id` permet d'identifier de manière unique chaque enregistrement.

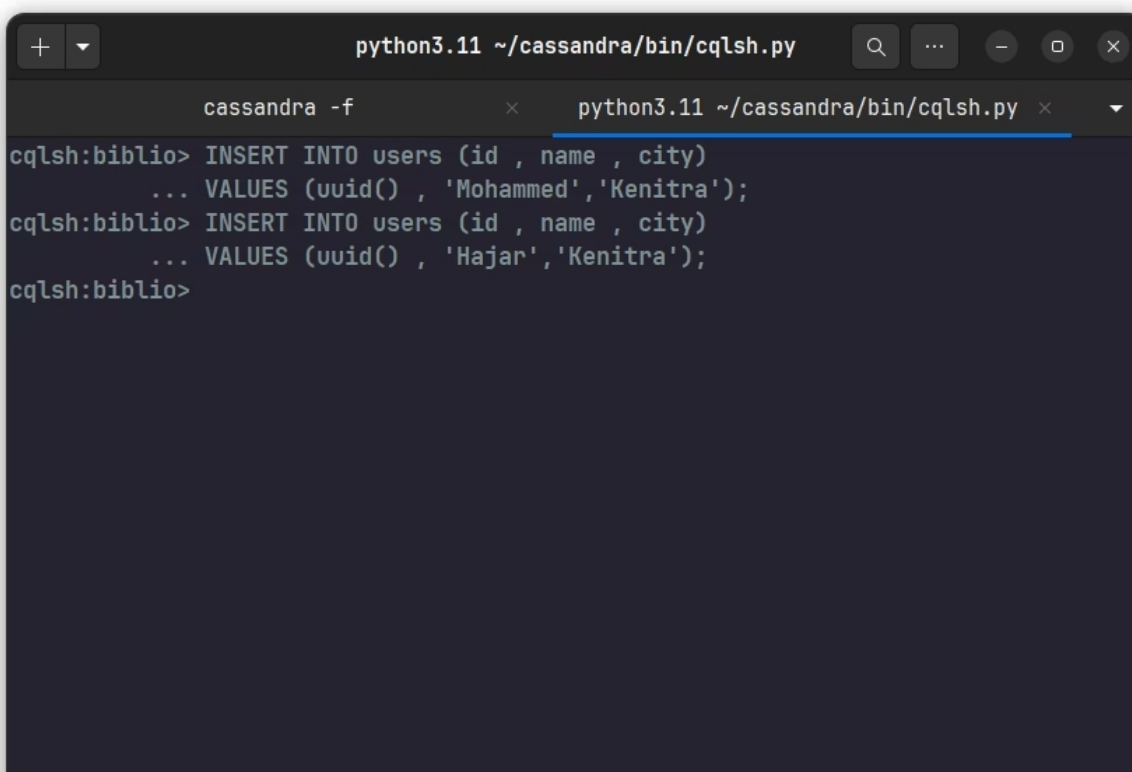


```
python3.11 ~/cassandra/bin/cqlsh.py
cassandra -f x python3.11 ~/cassandra/bin/cqlsh.py x
cqlsh:biblio> CREATE TABLE users (
... id UUID PRIMARY KEY,
... name TEXT,
... city TEXT
... );
cqlsh:biblio>
```

7. Manipulation des données

7.1 Insertion de données

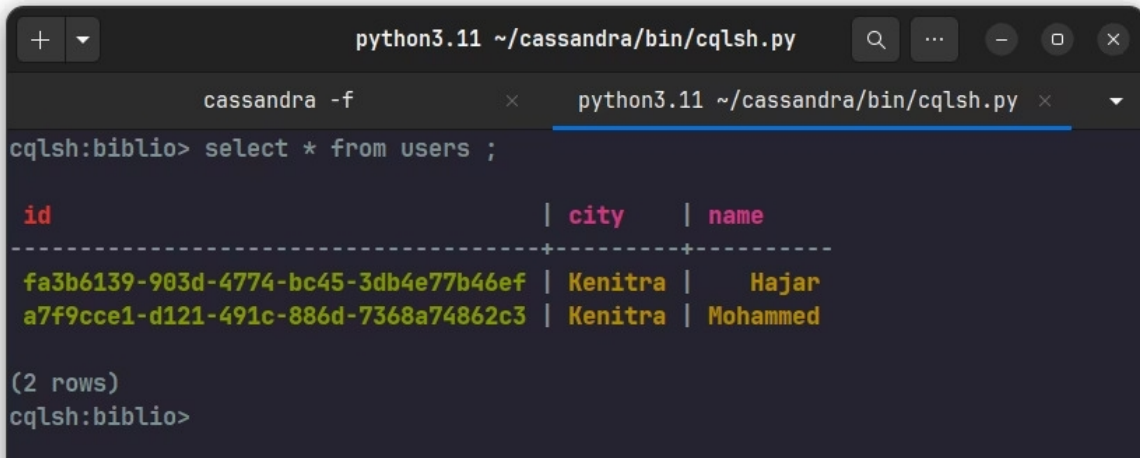
Une fois la table créée, des enregistrements sont ajoutés à l'aide de la commande `INSERT INTO`. Chaque utilisateur reçoit un identifiant généré automatiquement grâce à la fonction `uuid()`, ainsi que son nom et sa ville de résidence.



```
python3.11 ~/cassandra/bin/cqlsh.py
cassandra -f x python3.11 ~/cassandra/bin/cqlsh.py x
cqlsh:biblio> INSERT INTO users (id , name , city)
... VALUES (uuid() , 'Mohammed','Kenitra');
cqlsh:biblio> INSERT INTO users (id , name , city)
... VALUES (uuid() , 'Hajar','Kenitra');
cqlsh:biblio>
```

7.2 Consultation des données

La commande `SELECT * FROM users` permet d'afficher l'ensemble des enregistrements présents dans la table. Cette opération est utilisée pour vérifier que les données ont été correctement enregistrées dans Cassandra.



```
python3.11 ~/cassandra/bin/cqlsh.py
cassandra -f x python3.11 ~/cassandra/bin/cqlsh.py x
cqlsh:biblio> select * from users ;

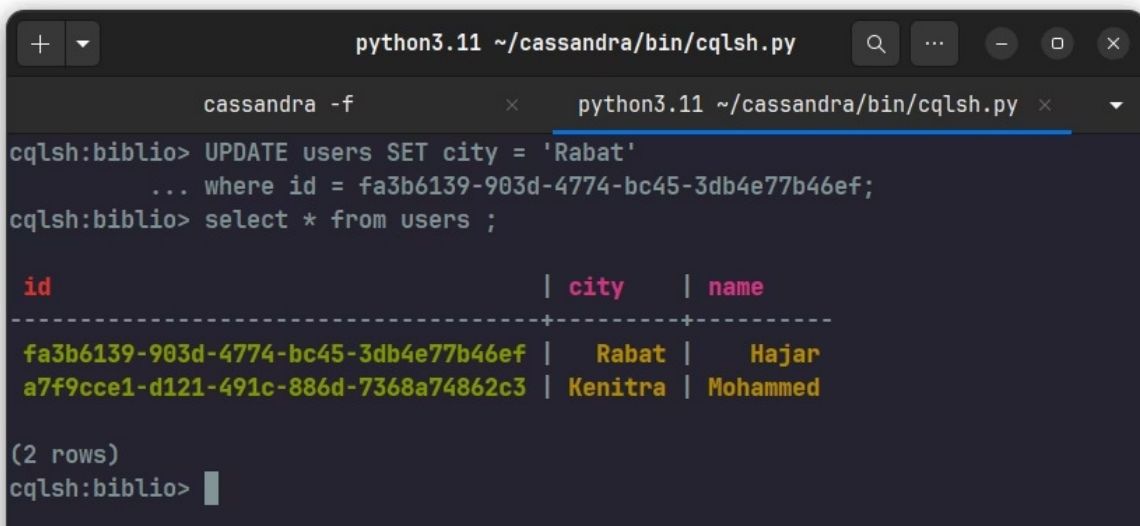
id | city | name
-----+-----+-----
fa3b6139-903d-4774-bc45-3db4e77b46ef | Kenitra | Hajar
a7f9cce1-d121-491c-886d-7368a74862c3 | Kenitra | Mohammed

(2 rows)
cqlsh:biblio>
```

Le résultat obtenu montre que Cassandra a correctement stocké les données et qu'elles peuvent être consultées à tout moment via le langage CQL (Cassandra Query Language). Cette étape valide le bon fonctionnement du Keyspace, de la table et des opérations de manipulation des données.

7.3 Mise à jour

Cassandra permet de modifier les données existantes à l'aide de la commande `UPDATE`. Dans cette étape, la ville associée à un utilisateur est mise à jour en utilisant son identifiant unique (id) comme clé de recherche. Cette opération permet de modifier certaines informations sans affecter les autres attributs de l'enregistrement.



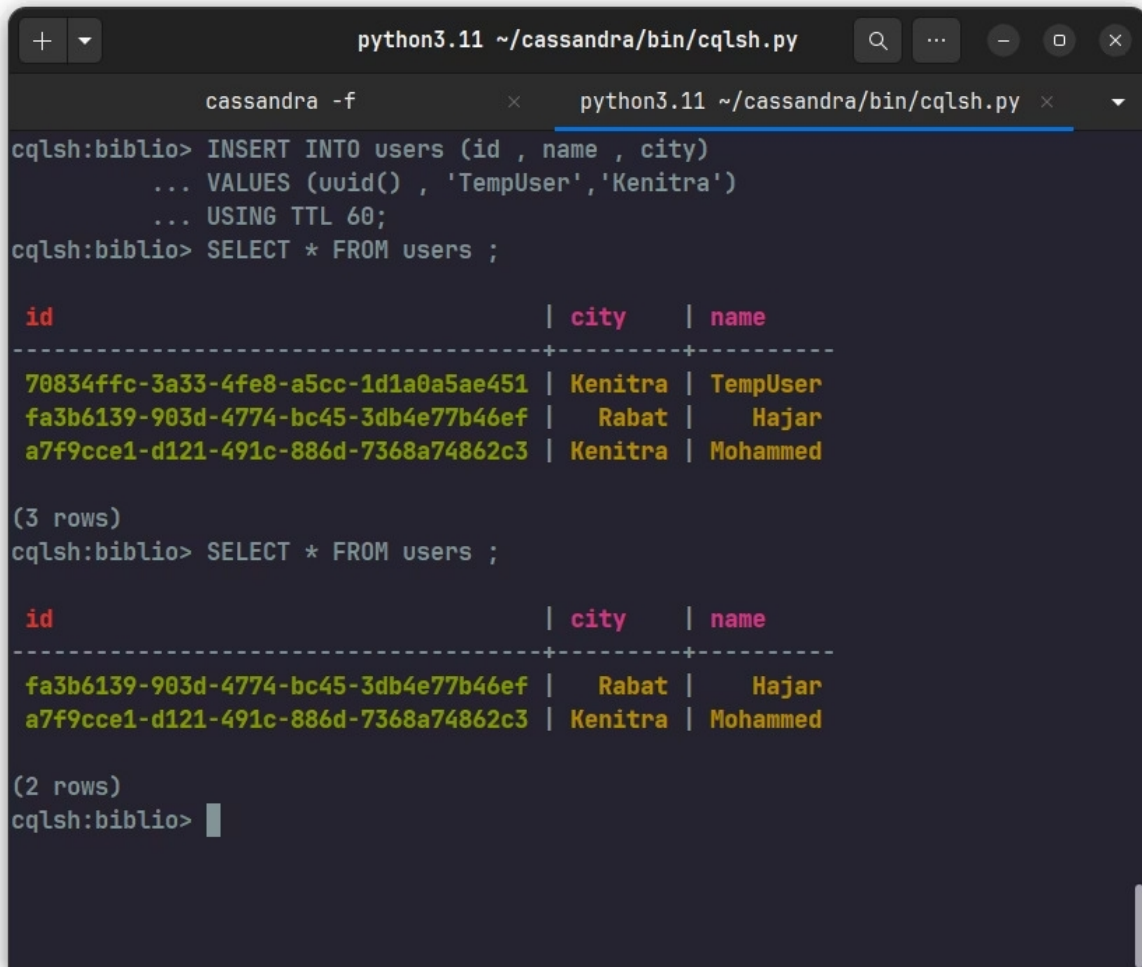
```
python3.11 ~/cassandra/bin/cqlsh.py
cassandra -f x python3.11 ~/cassandra/bin/cqlsh.py x
cqlsh:biblio> UPDATE users SET city = 'Rabat'
... where id = fa3b6139-903d-4774-bc45-3db4e77b46ef;
cqlsh:biblio> select * from users ;

id | city | name
-----+-----+-----
fa3b6139-903d-4774-bc45-3db4e77b46ef | Rabat | Hajar
a7f9cce1-d121-491c-886d-7368a74862c3 | Kenitra | Mohammed

(2 rows)
cqlsh:biblio>
```

7.4 Données temporaires (TTL)

Cassandra permet de définir une durée de vie (Time To Live - TTL) pour certains enregistrements. Lorsqu'un TTL est spécifié lors de l'insertion des données, l'enregistrement est automatiquement supprimé après l'expiration du délai défini. Cette fonctionnalité est particulièrement utile pour gérer les données temporaires, les sessions utilisateur ou les informations à durée limitée.



```
python3.11 ~/cassandra/bin/cqlsh.py
cassandra -f x python3.11 ~/cassandra/bin/cqlsh.py x
cqlsh:biblio> INSERT INTO users (id , name , city)
... VALUES (uuid() , 'TempUser','Kenitra')
... USING TTL 60;
cqlsh:biblio> SELECT * FROM users ;

id | city | name
-----+-----+-----
70834ffc-3a33-4fe8-a5cc-1d1a0a5ae451 | Kenitra | TempUser
fa3b6139-903d-4774-bc45-3db4e77b46ef | Rabat | Hajar
a7f9cce1-d121-491c-886d-7368a74862c3 | Kenitra | Mohammed

(3 rows)
cqlsh:biblio> SELECT * FROM users ;

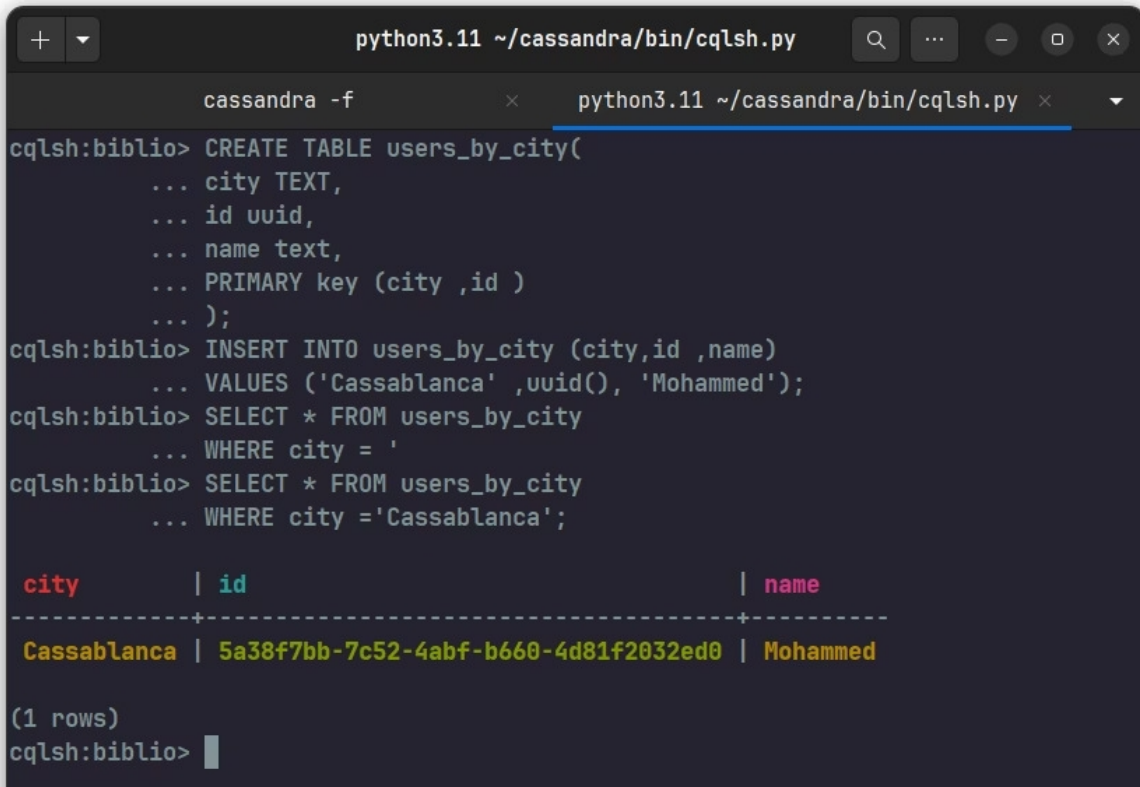
id | city | name
-----+-----+-----
fa3b6139-903d-4774-bc45-3db4e77b46ef | Rabat | Hajar
a7f9cce1-d121-491c-886d-7368a74862c3 | Kenitra | Mohammed

(2 rows)
cqlsh:biblio> 
```

Un nouvel utilisateur nommé **TempUser** a été inséré dans la table `users` avec un TTL de **60 secondes**. Une première consultation montre que l'enregistrement est bien présent dans la table. Après l'expiration du délai, une nouvelle consultation confirme que l'utilisateur a été automatiquement supprimé par Cassandra, tandis que les autres données restent inchangées.

8. Modélisation orientée requêtes

Contrairement aux bases de données relationnelles, Cassandra adopte une approche de modélisation orientée requêtes (Query-Based Modeling). Les tables sont conçues en fonction des requêtes les plus fréquentes afin d'optimiser les performances de lecture. Dans cette étape, une table `users_by_city` est créée pour permettre la recherche rapide des utilisateurs à partir de leur ville.



```
python3.11 ~/cassandra/bin/cqlsh.py
cassandra -f x python3.11 ~/cassandra/bin/cqlsh.py x
cqlsh:biblio> CREATE TABLE users_by_city(
... city TEXT,
... id uuid,
... name text,
... PRIMARY key (city ,id )
... );
cqlsh:biblio> INSERT INTO users_by_city (city,id ,name)
... VALUES ('Cassablanca' ,uuid(), 'Mohammed');
cqlsh:biblio> SELECT * FROM users_by_city
... WHERE city = '
cqlsh:biblio> SELECT * FROM users_by_city
... WHERE city ='Cassablanca';

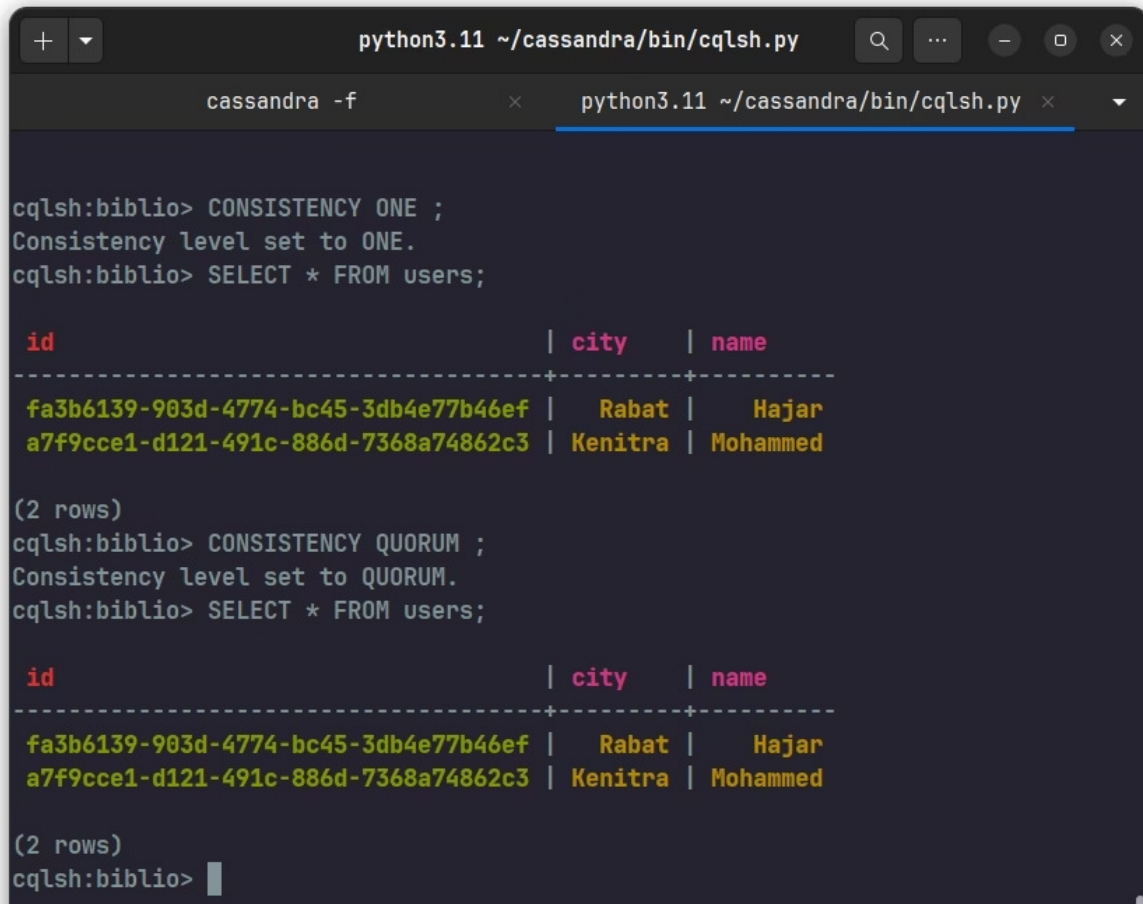
city | id | name
-----+-----+-----
Cassablanca | 5a38f7bb-7c52-4abf-b660-4d81f2032ed0 | Mohammed

(1 rows)
cqlsh:biblio>
```

Une table partitionnée par la colonne `city` a été créée. Un utilisateur a ensuite été inséré dans la ville de **Casablanca**. La requête de recherche basée sur la ville retourne correctement les informations enregistrées, démontrant l'efficacité de ce modèle de stockage.

9. Niveaux de cohérence

Cassandra permet de configurer différents niveaux de cohérence (Consistency Level) afin de contrôler le compromis entre disponibilité, performance et cohérence des données. Dans cette expérience, deux niveaux de cohérence sont utilisés : ONE et QUORUM.



```
python3.11 ~/cassandra/bin/cqlsh.py
cassandra -f x python3.11 ~/cassandra/bin/cqlsh.py x
cqlsh:biblio> CONSISTENCY ONE ;
Consistency level set to ONE.
cqlsh:biblio> SELECT * FROM users;

id | city | name
-----+-----+-----
fa3b6139-903d-4774-bc45-3db4e77b46ef | Rabat | Hajar
a7f9cce1-d121-491c-886d-7368a74862c3 | Kenitra | Mohammed

(2 rows)
cqlsh:biblio> CONSISTENCY QUORUM ;
Consistency level set to QUORUM.
cqlsh:biblio> SELECT * FROM users;

id | city | name
-----+-----+-----
fa3b6139-903d-4774-bc45-3db4e77b46ef | Rabat | Hajar
a7f9cce1-d121-491c-886d-7368a74862c3 | Kenitra | Mohammed

(2 rows)
cqlsh:biblio>
```

Après avoir défini le niveau de cohérence ONE, la requête de consultation retourne correctement les données enregistrées. Le même résultat est obtenu avec le niveau QUORUM, confirmant que les informations sont disponibles et cohérentes dans le cluster.

Remarque pédagogique

Dans cette configuration locale à un seul nœud, la différence entre ONE et QUORUM n'est pas visible. Ces niveaux prennent toute leur importance dans un cluster Cassandra distribué composé de plusieurs nœuds et répliques.